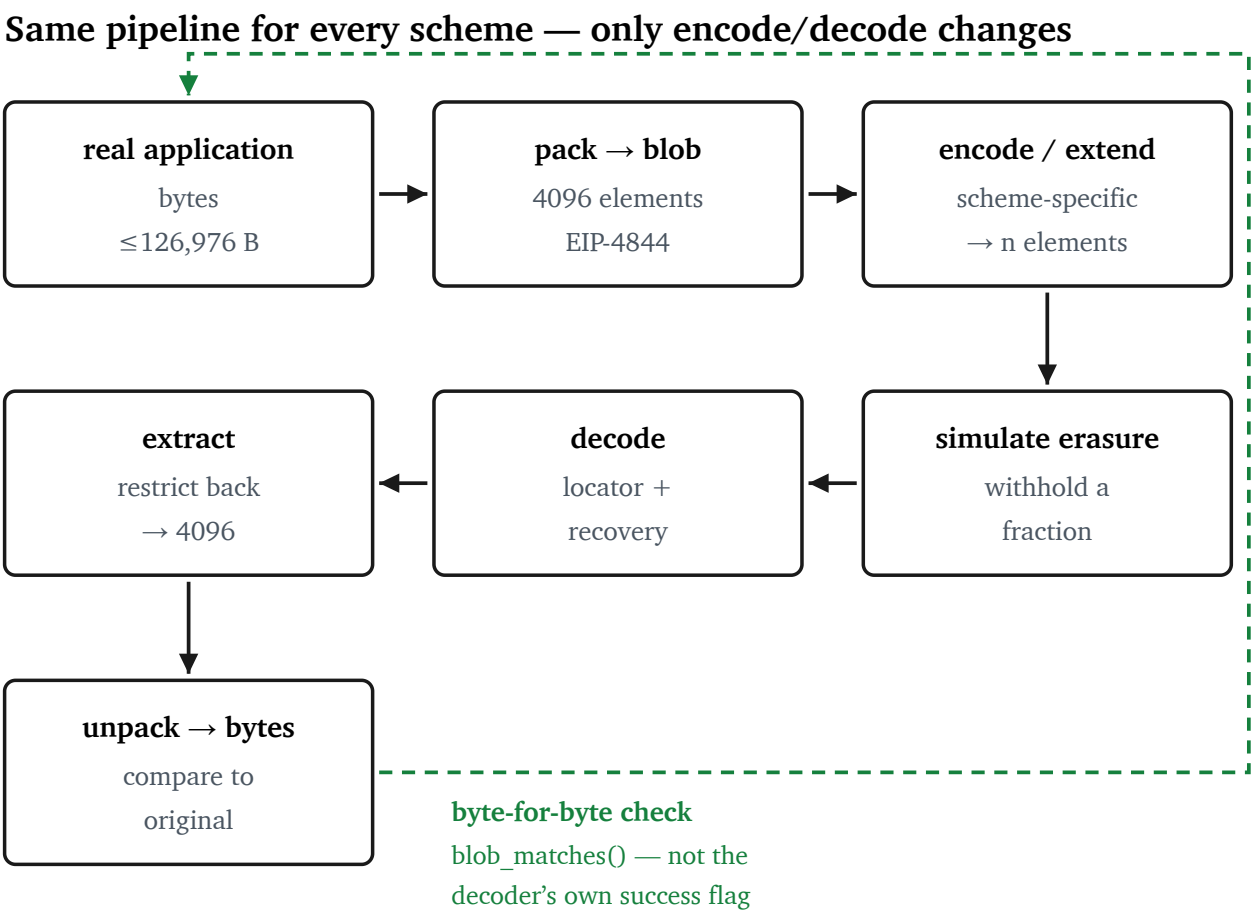
Five figures: the shared experimental pipeline, how each scheme builds its codeword, and the two block-circulant figures that explain why an optimal code can still decode poorly. Everything here is either a pipeline the report has to describe anyway, or the structure behind a measured result. Earlier drafts also carried worked numeric examples and algorithm flowcharts; those were teaching aids rather than results and have been dropped.

Figures are drawn from this project's own code (`1D_RS_Galois.py`, `2D_RS_Galois.py`, `BCcodeEthereum.py`). The BC figures follow the published construction — Sasidharan, Viterbo & Hoang Dau, "*Block Circulant Codes with Application to Decentralized Systems,*" IEEE Trans. Commun. 73(10), 2025 (arXiv:2406.12160) — since that is what the implementation is meant to realise.

Experimental pipeline

Every number in the benchmark report comes from running this loop once per trial. The blob is fixed and the erasure rate is swept; at each rate the codeword is corrupted at randomly chosen positions and handed to the scheme's own decoder.



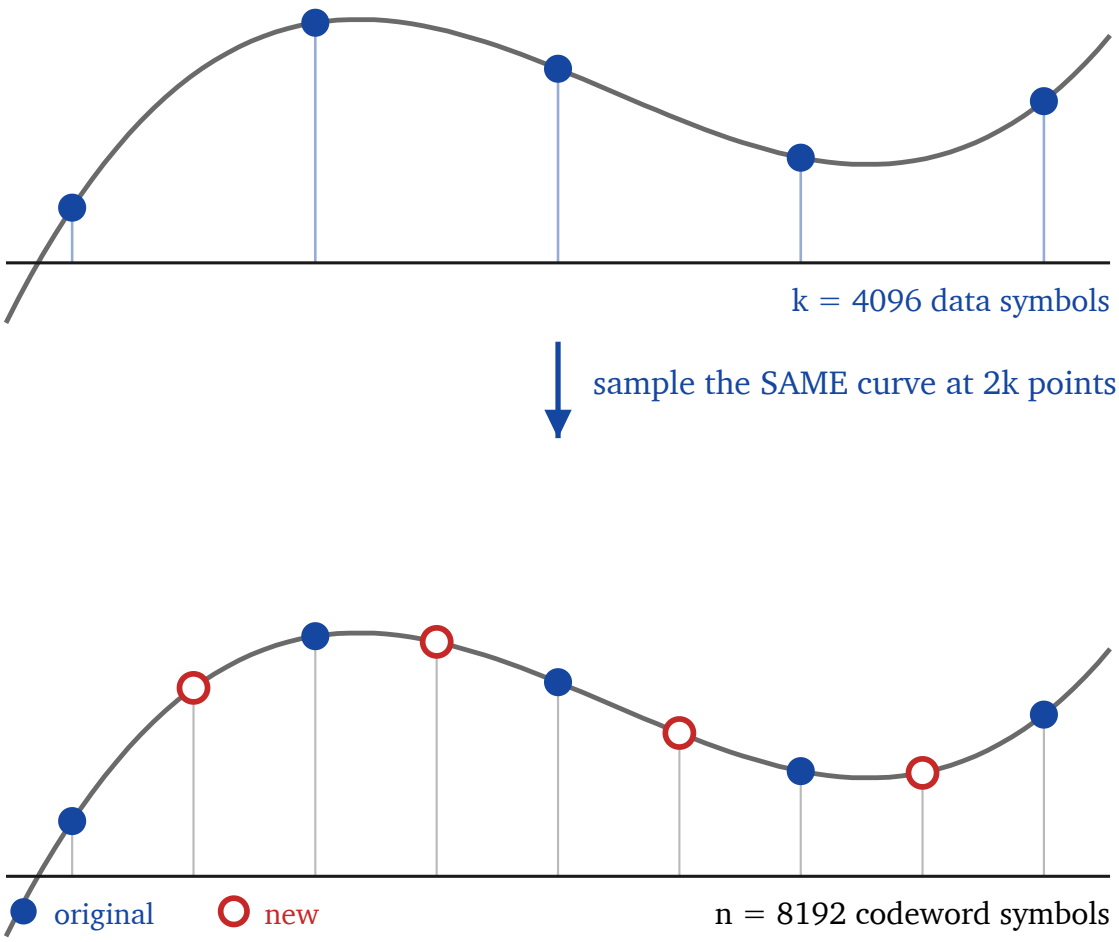
The measurement pipeline. Real application bytes are packed into one EIP-4844 blob (4096 field elements, 31 usable bytes each), the scheme under test expands that blob into a longer codeword, a fraction of the codeword is withheld to stand in for samples peers failed to serve, the decoder tries to rebuild it, and the result is unpacked back to bytes. Only *encode/extend* and *decode* differ between schemes; the input, the erasure model and the test of success are byte-identical for all of them, which is what makes a difference in the results attributable to the code rather than to the setup. The dashed return path is the success test itself: the recovered bytes must equal the original bytes exactly. It deliberately does not ask the decoder whether it succeeded — one decoder was found reporting success on data it had silently corrupted, and only this comparison caught it.

How each scheme builds its codeword

The two Reed-Solomon variants share one idea applied at different dimensions. Both figures lead with that idea rather than with the transform steps, because the mechanism is easier to follow once the purpose is clear.

The blob IS a polynomial, sampled

k data symbols = its values at k points

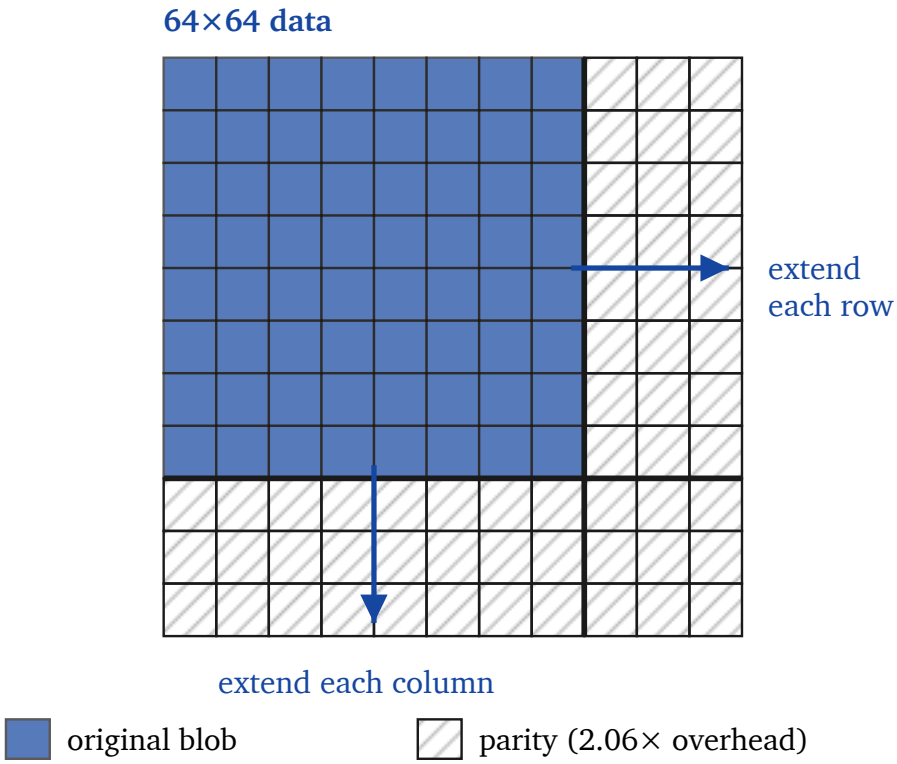


**Any k of the n samples pin down the curve —
so any k recover every original symbol.**

1D-RS. The blob's k field elements are not raw bytes to be copied; they are the *values of a single polynomial* sampled at k points. Encoding finds that polynomial and samples it at n = 2k points instead. Because a degree-<k polynomial is pinned down by any k of its values, any k surviving symbols reconstruct the curve and hence every original symbol. 2× overhead, and the reason 1D-RS reaches its theoretical limit exactly.

The same trick, twice

rows 64→96, columns 64→88: overall rate 0.485



Every row and every column is now its own 1D-RS codeword, so a lost cell can be repaired from its row or its column — whichever survives.

2D-RS. The blob is reshaped into a 64×64 grid and the 1D-RS step is applied twice: once to extend every row, then once to extend every column of the result. Every row and every column of the outcome is independently a 1D-RS codeword, so a lost cell can be repaired from whichever of its two lines has enough survivors, which is what lets the iterative decoder cascade. The grid drawn here is schematic. The real extension is 64→96 along rows and 64→88 down columns, rather than the 64→128 of earlier drafts: a product code's rate is the *product* of its component rates, so doubling both axes costs 4× storage and made the comparison against 2×-overhead schemes unfair. Component rates 64/96 and 64/88 give 0.485 overall at 2.06× storage — the closest to rate 0.5 reachable in this field, since the length-90 NTT an even 90×90 split would need does not exist here.

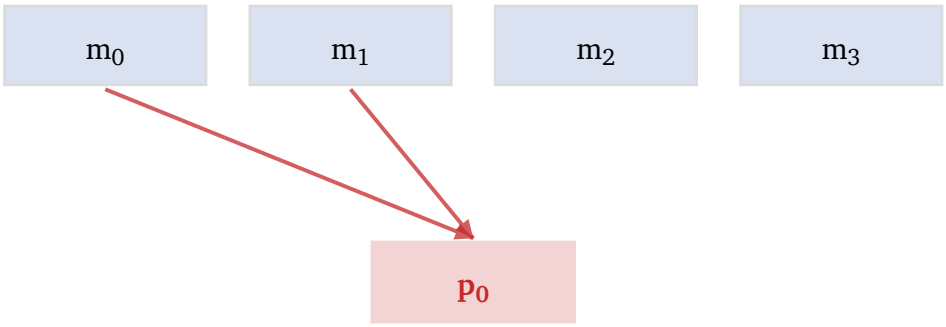
Block-circulant structure

Block-circulant codes replace one global polynomial with many small local ones. How much they overlap is the parameter that decides whether the resulting code is any good.

Full overlap makes the code optimal

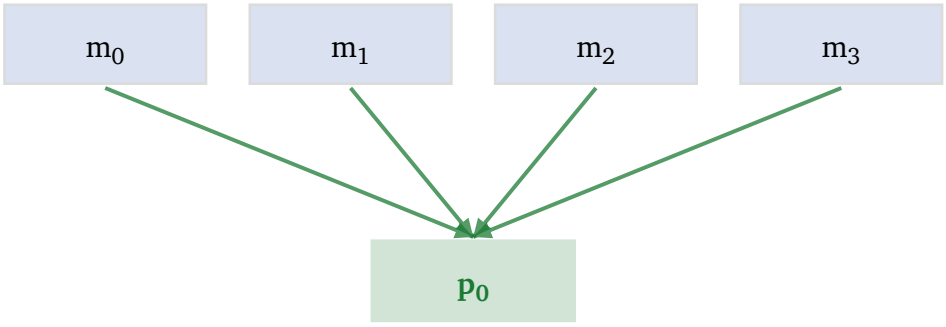
how many information blocks each parity block sees

λ = 2 (three of the five rows swept)



Parity block 0 is computed from 2 of the μ blocks.

λ = μ (ω2048 λ2 and ω1024 λ4)



Parity block 0 is computed from every block.

d = λρ + 1, up to 4097 = n − k + 1 (Singleton) at λ = μ.

Overlap decides the code's guarantee. Minimum distance is $d = \lambda\rho + 1$, so the more information blocks each parity block depends on, the more erasures the code is *guaranteed* to survive. At $\lambda = \mu$ every parity block depends on every information block and d reaches $4097 = n - k + 1$, the Singleton bound: as a code that is MDS, exactly as good as Reed-Solomon. Two of the five rows swept are like that — ω2048 λ2, where $\mu = 2$, and ω1024 λ4. The λ=2 rows at $\mu = 4$ and 8 and the λ=3 row are not, with worst-case guarantees of 0.250, 0.125 and 0.375. (The project's first parameterisation, $\omega = \rho = 2$ with $\mu = 2048$, had $d = 5$: guaranteed to correct only four erasures in an 8192-symbol codeword.)

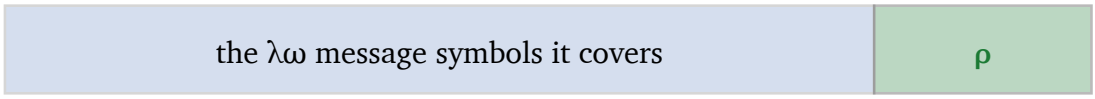
Why block-circulant tolerance is limited

Whatever the code guarantees, the decoder stops early, and for the same reason in every row: it never looks beyond a single local block. Even the two MDS rows — where any 4096 surviving symbols determine the blob, exactly as for 1D-RS — stop recovering near a third and a fifth of the codeword respectively.

Whatever the code, the decoder is local

what one local block can see, and what it can fix

one local block's support



no = ρ + λω

It solves when its support holds at most ρ erasures.

At erasure rate x, about x·no of them are missing.

λωx + ρx ≤ ρ

so it gives out at x = ρ/no = 1/(1+λ) when ρ = ω.

row	no	ρ/no	solve cost
ω2048 λ2	6144	**0.333**	2048×2048
ω1024 λ2	3072	**0.333**	1024×1024
ω512 λ2	1536	**0.333**	512×512
ω1024 λ3	4096	**0.250**	1024×1024
ω1024 λ4	5120	**0.200**	1024×1024

The gap is the decoder, not the code.

Even the MDS rows (ω2048 λ2, ω1024 λ4) stop here;

measured thresholds sit just above, at 0.336–0.349, 0.255, 0.204.

Why the measured tolerance is far below 0.50 anyway. `DecCBC[2]` solves one block at a time, and a block succeeds only when its own support — its ρ parity symbols plus the λω message symbols it covers — holds at most ρ erasures. Solving $\lambda\omega x + \rho x \le \rho$ gives $x \le \rho/no = 1/(1+\lambda)$ when $\rho = \omega$, and the measurements land just above it: 0.336–0.349 for $\lambda = 2$, 0.255 for $\lambda = 3$ and 0.204 for $\lambda = 4$, the small excess coming from the cascade across blocks. That limit holds even for the two MDS rows, whose code alone could survive half the codeword. Because the local system is a Vandermonde matrix in distinct evaluation points it can be solved in ρ^2 rather than ρ^3 time — which is what makes $\omega = 1024$ and 2048 measurable at all.